6
Features, Regularization, and Validation

This chapter introduces three new concepts: constructed features, regularization, and validation. Each can be used on its own. For example, regularization can be used without constructing new features, and validation is used for hyper-parameters other than those in regularization. However, taken in this order, each motivates the next.

Constructed Features

Both linear regression and logistic regression learned linear functions. But, what if we want to learn a non-linear relationship between the label and the features? For instance, Figure 2.3 shows the fit of f as various polynomial functions. Or we might try to build a a discriminant, $\widetilde{f}$, that generates a non-linear decision boundary.

Provided that the resulting hypothesis space is still linear *in the weights*, the same techniques we used in the previous chapters will work. For instance, if we wish to make $f(\vec{x})$ a quadratic function of $\vec{x}$ and $n = 2$ (that is, there are two features, or $\vec{x}$ is 2-dimensional), the function takes the following form.

$$\begin{aligned} f(\vec{x}) &= w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1 x_2 + w_5 x_2^2 \\ &= \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \\ w_4 \\ w_5 \end{bmatrix}^\intercal \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_1 x_2 \\ x_2^2 \end{bmatrix} \end{aligned} \tag{6.1}$$

Thus, just as we handled the intercept, w_0, by adding an additional feature that is always 1, we can handle other terms by adding additional features.

Regression Example

To return to the example at the end of Chapter 4, we could learn to predict the land speed of a hippopotamus as a quadratic function of its girth and wiggle rate. We transform the data by adding three extra columns: girth-squared, girth-times-wiggle-rate, and wiggle-rate-squared:

$$\boldsymbol{X} = \begin{bmatrix} 1.00 & 3.49 & 0.41 & 12.18 & 1.43 & 0.17 \\ 1.00 & 2.39 & 0.34 & 5.71 & 0.81 & 0.12 \\ 1.00 & 3.28 & 0.14 & 10.76 & 0.46 & 0.02 \\ 1.00 & 3.10 & 0.31 & 9.61 & 0.96 & 0.10 \\ 1.00 & 3.01 & 0.27 & 9.06 & 0.81 & 0.07 \\ 1.00 & 3.11 & 0.53 & 9.67 & 1.65 & 0.28 \\ 1.00 & 2.57 & 0.53 & 6.60 & 1.36 & 0.28 \\ 1.00 & 3.07 & 0.38 & 9.42 & 1.17 & 0.14 \\ 1.00 & 2.54 & 0.36 & 6.45 & 0.91 & 0.13 \\ 1.00 & 3.64 & 0.05 & 13.25 & 0.18 & 0.00 \end{bmatrix} \quad \boldsymbol{Y} = \begin{bmatrix} 12.01 \\ 8.49 \\ 17.81 \\ 25.67 \\ 22.89 \\ 26.09 \\ 18.53 \\ 24.47 \\ 15.76 \\ 8.98 \end{bmatrix}$$

Figure 6.1: Data from Table 2.1 with three extra constructed features (columns): $x_3 = x_1^2, x_4 = x_1 x_2, x_5 = x_2^2$.

The new columns do not represent new measurements, but combinations of existing measurements: The fourth column's values are the squares of second column's values. The fifth column's values are the multiplication of the corresponding values in the second and third columns. And the sixth column's values are the squares of the third column's values.

Each new column corresponds to a **constructed feature**. This feature's value is not in the original dataset, but can be computed from the features (of the same example) in the dataset. The original data (the second and third columns above) are sometimes called the **raw features**. These are the values as provided. However, the distinction is somewhat arbitrary. For instance, if our data had the weight of each hippopotamus, these values are assuredly not measured directly either. Rather, the compression of a spring (or similar) was probably measured and converted into an estimated weight. If an ideal spring were assumed (Hooke's law), this would be a linear relationship, but with a more realistic relationship between depression and weight, the recorded "raw" feature (weight) is a non-linear function of the truly measured quantity (compression distance).

Key point
One person's constructed features are another person's raw features.

When necessary to distinguish between raw features and constructed features, we might let ϕ represent the transform from raw features to constructed features: x is the raw feature vector and $\phi(x)$ is the corresponding constructed feature vector.[1] With this notation,

[1] phi for phiture

$f(\vec{x}) = \vec{w}^{\mathsf{T}}\phi(\vec{x})$. However, most commonly, we do not make a distinction. Anything that can be done with a raw feature can be done with a constructed feature, assuming that the method for construction is known before learning starts. Therefore, in this text we do not use different notation for the constructed feature and the raw features. We assume that all features have already been transformed into the constructed features and specifically the matrix $\boldsymbol{X}$ has already been transformed to be the constructed features, as above. For example, we could carry out linear least squares regression as at the end of Chapter 4 on this new dataset (with added constructed features) in exactly the same way:

$$\vec{c} = \begin{bmatrix} 180.70 \\ 545.70 \\ 63.51 \\ 1666.48 \\ 188.37 \\ 25.62 \end{bmatrix} \quad \boldsymbol{A} = \begin{bmatrix} 10.00 & 30.20 & 3.32 & 92.72 & 9.75 & 1.31 \\ 30.20 & 92.72 & 9.75 & 289.11 & 29.06 & 3.82 \\ 3.32 & 9.75 & 1.31 & 29.06 & 3.82 & 0.56 \\ 92.72 & 289.11 & 29.06 & 914.34 & 87.85 & 11.31 \\ 9.75 & 29.06 & 3.82 & 87.85 & 11.31 & 1.63 \\ 1.31 & 3.82 & 0.56 & 11.31 & 1.63 & 0.25 \end{bmatrix} \tag{6.2}$$

Note that A is now 6-by-6 because there are 6 features (including the "constant feature"). The final weight vector is

$$\vec{w} = \boldsymbol{A}^{-1}\vec{c} = \begin{bmatrix} -401.22 \\ 271.80 \\ 95.69 \\ -43.79 \\ -27.03 \\ -7.20 \end{bmatrix} \tag{6.3}$$

And the learned function (in terms of the original raw features) is

$$f(\vec{x}) = -401.22 + 271.80x_1 + 95.69x_2 - 43.79x_1^2 - 27.03x_1x_2 - 7.20x_2^2 \tag{6.4}$$

Figure 6.2 compares this fit (a quadratic fit) to the linear fit that just uses the raw features from Chapter 4. Note that the lower surface is linear, but in a 5-dimensional space (of which the data only occupy a 2-dimensional manifold because three of the dimensions are functions of the other two). When plotted back in terms of the original (raw) features, the prediction surface is quadratic.

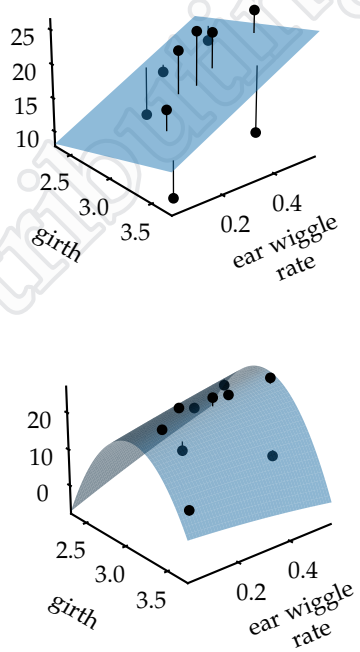

Figure 6.2: Plots of the fit f to the data from Table 2.1. (top) linear fit; (bottom) quadratic fit

Classification Example

The same can be done for classification, constructing new features so that the discriminant is a non-linear function of the original features: $\widetilde{f}(\vec{x}) = \vec{w}^{\mathsf{T}}\phi(\vec{x})$. Again, we generally do not specifically note the transformation of ϕ and instead just assume that every $\vec{x}$ has been

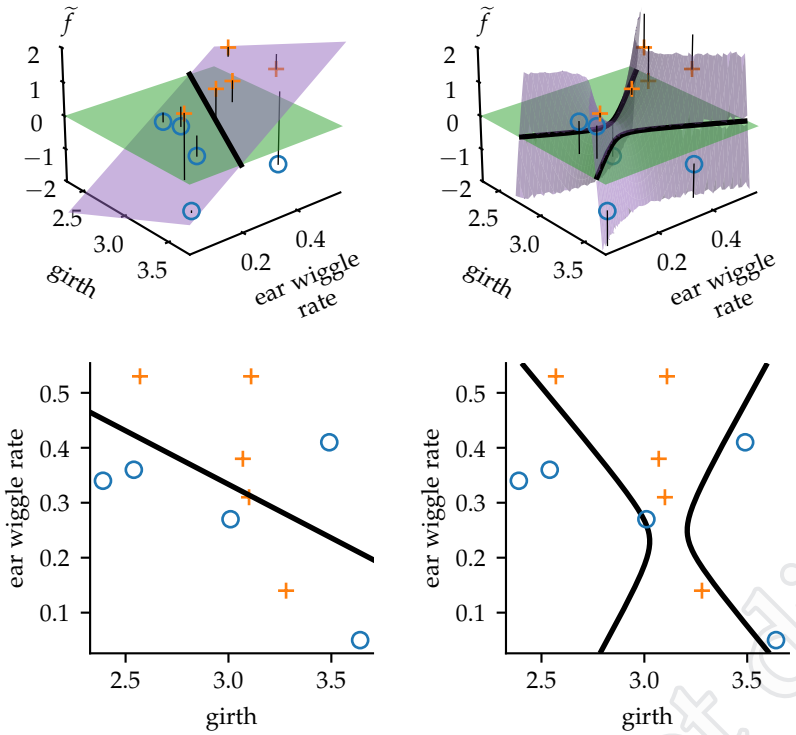

Figure 6.3: Plots of the fit $\widetilde{f}$ to the data from Table 5.1. (left) linear fit; (right) quadratic fit; (top) violet discriminant with green decision threshold at $\widetilde{f}$=0; (bottom) decision boundary only

replaced with the corresponding $\phi(\vec{x})$ (in training and testing). Figure 6.3 shows the discriminant and resulting decision boundaries when fit using logistic regression with linear (no transformation) and quadratic features for the hippopotamus aggression data of Table 5.1

Limitations

Does this mean with a suitable transformation of the data, we can use linear methods from the previous chapters for any hypothesis space? No. Although functions of the forms

$$f(\vec{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 (x_1 x_2^3) \tag{6.5}$$

$$f(\vec{x}) = w_0 + w_1 \sin(x_1) + w_2 e^{x_1 + x_2} \tag{6.6}$$

(as examples) can be learned easily using the techniques from the previous chapters because they are *linear in* $\vec{w}$, functions parameter-

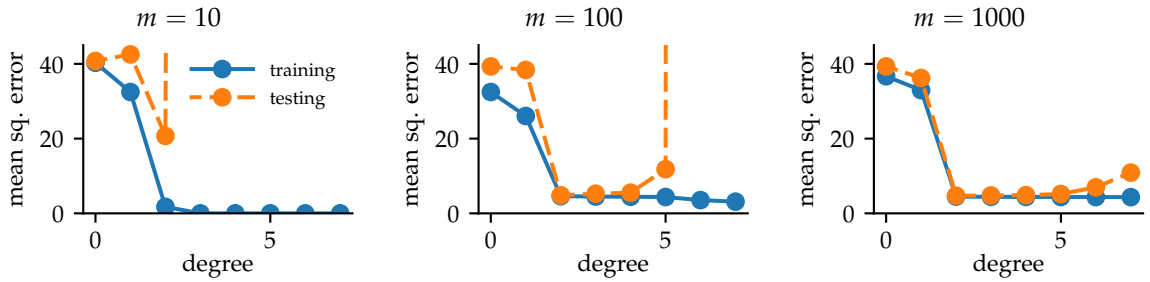

Figure 6.4: Training and testing errors as a function of the degree of the constructed polynomial features for least squared regression on the data from Table 2.1. Testing data are 10,000 points drawn from the same distribution as the training data. (left) 10 training examples (same as Table 2.1); (middle and right) 100 and 1000 training points, drawn from the same distribution as Table 2.1.

ized with forms

$$f(\vec{x}) = w_0 + w_1 \log(w_2 + x_1) \tag{6.7}$$

$$f(\vec{x}) = w_0 \sin(w_1 x_1) + w_2 \sin(w_3 x_2) \tag{6.8}$$

(as examples) are not linear in $\vec{w}$ and cannot be converted into forms that have a linear relationship with some set of parameter. Therefore, they correspond to hypothesis spaces that cannot be learned with techniques from the previous chapters.

Key point
A linear method is *linear* because the learned function form is *linear in* $\vec{w}$. It need not be linear in the (raw) $\vec{x}$ values.

More significantly, with this new-found power to increase the complexity of the hypothesis space comes the new-found risk of overfitting. Consider Figure 6.3. The quadratic fit seems overly specific to these data points. While the linear fit might be missing some nuances about the relationship between physical attributes and aggressiveness in hippopotamuses, it is probably a better choice for future data. We can see a similar effect in Figure 6.2.

Figure 6.4 shows the effect of increasing the number of constructed features for the hippopotamus running speed regression task. The number of constructed features rises rapidly with the degree of the polynomial. The corresponding extra degrees of freedom in the weight vector allow for overfitting more easily. On the left (m=10) we can see a second-order fit (Figure 6.2, bottom) does improve the training and testing error. Increasing the degree of the polynomial beyond 2 causes the training error to go to zero (fitting all 10 points perfectly), but the resulting function overfits badly, and the testing error is very large. The middle and right plots show that with more training data, a higher degree polynomial can be estimated without overfitting. But, overfitting still happens if the polynomial's degree is large enough.

Therefore, while it is tempting to add many features to allow for flexibility in the learned function, doing so can easily cause overfit-

ting. The appropriate number of extra parameters (or even which features should be added) cannot be determined from the training set. Adding extra features always improves the training accuracy. Whether it decreases the testing accuracy (that is, causes overfitting) is unknown. Further, the number or type of extra features that can be tolerated (or gainfully exploited) is determined in part by the size of the training set, m. Thus, it is a complicated interplay among the task at hand, the learning algorithm, and the amount of training data.

Further information
The flexibility of the learned function is often called the capacity of the hypothesis space.

Regularization

For linear methods, when the number of features exceeds the number of data points (that is, $n > m$), the solution is underdetermined: There are multiple possible weight vectors that all yield the minimum possible error. For linear regression, this manifests as the matrix $\boldsymbol{X}^\intercal\boldsymbol{X}$ being singular, and therefore we cannot invert it as necessary (see Equation 4.13). Even if there are more examples than features, having too many weights (degrees of freedom) is problematic, as seen above. To select among these multiple possible weight vectors, we need a secondary criterion.

A common secondary criterion is the "simplicity" of the resulting learned function. Simplicity is somewhat in the eye of the beholder, so there is not a single definition. However, for the moment, we might assume that functions with smaller weights (coefficients) are simpler. Why? A straight-forward reason is that they have smaller derivatives: The weights are the partial derivatives for a linear function. There are many ways of quantifying the size of the weights, but the squared length of the weight vector is a reasonable one: $\sum_{j=1}^{n} w_n^2 = \vec{w}^\intercal\vec{w}$.

Ockham's razor[2] implores us not to make things more complex than necessary. It is a reasonable guiding principle, but tricky to apply because of the "than necessary" clause: What is necessary? With enough features, we could perfectly predict all of the training examples. We might be tempted to use the secondary criterion (the simplicity of the learned function) to select only among those functions that fit all of the training examples. Yet, fitting all of the training data perfectly may be "more than necessary" and still result in overfitting.

Thus, as applied to machine learning, the secondary criterion of simplicity is not strictly subsidiary to that of fitting the training data. Rather the two are linearly combined so that the net goal is to minimize a combination the loss on the data and the complexity of the resulting function. The general form is to recast the total loss, L,

[2] Not invented by William of Ockham, but employed frequently by him. It is stated in many ways including *Numquam ponenda est pluralitas sine necessitate* or Never impose plurality without necessity.

as

$$L = \underbrace{\sum_{i=1}^{m} l(y_i, f(\vec{x}_i))}_{\text{loss on all training data}} + \underbrace{\lambda}_{\text{regularization strength}} \underbrace{R(\vec{w})}_{\text{regularizer}} . \tag{6.9}$$

This approach to machine learning is called **regularization**: A scalar constant called the regularization strength (λ) is multiplied by a measure of complexity called the regularizer (R) and this is added to the original objective. Note that the original objective (the training loss) is a function of both the training data and the weights ($\vec{w}$ is hidden inside f). However, the regularizer is only a function of the weights.

Different regularizers are possible. The most common is called ℓ_2 regularization which penalizes according to the sum of the squares (hence the "2") of the weights. This is the regularizer mentioned above:

$$R(\vec{w}) = \vec{w}^{\mathsf{T}}\vec{w} . \qquad \text{the } \ell_2 \text{ regularizer} \tag{6.10}$$

Ridge Regression

If we apply ℓ_2 regularization to linear least squares, our objective function is

$$L = \sum_{i=1}^{m} (y_i - \vec{x}_i{}^{\mathsf{T}}\vec{w})^2 + \lambda\vec{w}^{\mathsf{T}}\vec{w} . \tag{6.11}$$

This particular combination of loss (least squares) and regularizer (ℓ_2) is known as **ridge regression**. Deriving the learning equation follows similar steps as that of linear least squares:

$$\begin{aligned}
\vec{0} &= \nabla_{\vec{w}} \left(\sum_{i=1}^{m} (y_i - \vec{x}_i{}^{\mathsf{T}}\vec{w})^2 + \lambda\vec{w}^{\mathsf{T}}\vec{w}) \right) \\
&= \underbrace{-2 \sum_{i=1}^{m} \vec{x}_i (y_i - \vec{x}_i{}^{\mathsf{T}}\vec{w}) + 2\lambda\vec{w}}_{\text{see Equation 4.10}} \\
\vec{0} &= -\left(\sum_{i=1}^{m} \vec{x}_i y_i \right) + \left(\sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^{\mathsf{T}} \right) \vec{w} + \lambda\vec{w} \\
&= -\left(\sum_{i=1}^{m} \vec{x}_i y_i \right) + \left(\sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^{\mathsf{T}} + \lambda\boldsymbol{I} \right) \vec{w} \\
\left(\sum_{i=1}^{m} \vec{x}_i y_i \right) &= \left(\sum_{i=1}^{m} \vec{x}_i \vec{x}_i{}^{\mathsf{T}} + \lambda\boldsymbol{I} \right) \vec{w} \\
\vec{w} &= (\boldsymbol{A} + \lambda\boldsymbol{I})^{-1}\vec{c}
\end{aligned} \tag{6.12}$$

where $\vec{c} = \boldsymbol{X}^\intercal \boldsymbol{Y}$ and $\boldsymbol{A} = \boldsymbol{X}^\intercal \boldsymbol{X}$, just as in linear least squares (see Equation 4.12). The addition of some amount of the identity matrix, $\lambda \boldsymbol{I}$ causes the resulting matrix, $\boldsymbol{A} + \lambda \boldsymbol{I}$, be non-singular (and therefore invertible).

Most commonly, if there is a weight for the intercept term (w_0), that weight is not included in the regularizer. This coefficient is not related to the slope and therefore often not penalized. This means that

$$R(\vec{w}) = \vec{w}^\intercal \tilde{\boldsymbol{I}} \vec{w} \tag{6.13}$$

where $\tilde{\boldsymbol{I}}$ is an identity matrix, except with a 0 in the upper-left corner:

$$\tilde{\boldsymbol{I}} = \begin{bmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & & \vdots \\ 0 & 0 & 1 & \ddots & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 0 & \cdots & \cdots & 0 & 1 \end{bmatrix} \tag{6.14}$$

Redoing the derivation of Equation 6.12, we get the same formula, except $\boldsymbol{I}$ is replaced with $\tilde{\boldsymbol{I}}$:

$$\vec{w} = (\boldsymbol{A} + \lambda \tilde{\boldsymbol{I}})^{-1} \vec{c} \ . \tag{6.15}$$

Logistic Regression with Regularization

Regularization can be applied to any loss function. For instance, we can add ℓ_2 regularization to logistic regression and our total loss function becomes

$$L = \sum_{i=1}^{m} \left(-\ln \sigma(y_i \vec{x}_i{}^\intercal \vec{w}) \right) + \lambda \vec{w}^\intercal \vec{w} \tag{6.16}$$

(compare with Equation 5.12). Just like un-regularized logistic regression, we are forced to use numeric optimization to find the minimum of the loss function. We can again use gradient descent. The gradient is largely the same, except there is an extra term corresponding to the regularizer:

$$\nabla_{\vec{w}} L = \left(-\sum_{i=1}^{m} \left(1 - \sigma(y_i \vec{x}_i{}^\intercal \vec{w})\right) y_i \vec{x}_i \right) + 2\lambda \vec{w} \tag{6.17}$$

(compare with Equation 5.14).

Recall that the update rule for gradient descent is to *subtract* the gradient (times a step size) from the weight vector. For ℓ_2 regularized logistic regression, the update rule for gradient descent is, therefore,

$$\vec{w} \leftarrow \vec{w} + \underbrace{\eta \left(\sum_{i=1}^{m} \left(1 - \sigma(y_i \vec{x}_i{}^\intercal \vec{w})\right) y_i \vec{x}_i \right)}_{\text{same as for regular logistic regression}} \underbrace{-2\eta\lambda\vec{w}}_{\text{new term}} \tag{6.18}$$

This new term is a vector that points from the current weights toward the origin (it is in the direction of $-\vec{w}$). Therefore, on its own, it pulls the weights toward $\vec{0}$. For this reason, ℓ_2 regularization is often call **weight decay** when used with gradient descent, because the effect is to "decay" the weights toward zero. Of course, the original term from the loss on the data pulls the weights toward values that fit the data. It is the trade-off between these two that leads to a regularized solution.

Further information
While linear least squares regression combined with ℓ_2 regularization has its own name (ridge regression), for most other combinations of a base method with a type of regularization, there is no special name. Instead we generally refer to them as "X with ℓ_2 regularization" or "ℓ_2 regularized X."

Effect of Regularization

Figure 6.5 shows the effects of regularization for the running regression example example with m=100 examples (compare with the middle plot in Figure 6.4). Note that as we increase the regularization strength (λ) the training error gets worse across all numbers of features (degree of the polynomial). However, the testing error (the value we care about, but cannot be measured during training) generally gets closer to the training error. Small values of the regularization strength (λ=10^{-6} in this case) improve the testing accuracy when there are fewer features. Larger values for the regularization strength (λ=1 in this case) improve the testing accuracy when there are more features, but make the testing accuracy worse when there are fewer features. If the regularization strength is too large ($\lambda = 10^4$ in this case), the method underfits (is too smooth) and the testing and training errors are both large. If the regularization strength is too small, the method overfits (fits the noise as well as the signal).

Figure 6.6 shows how the training and testing errors vary with the regularization strength, when fixing the features. Note that the training error always decreases when the regularization strength is less. However, as we increase the regularization strength, the testing error

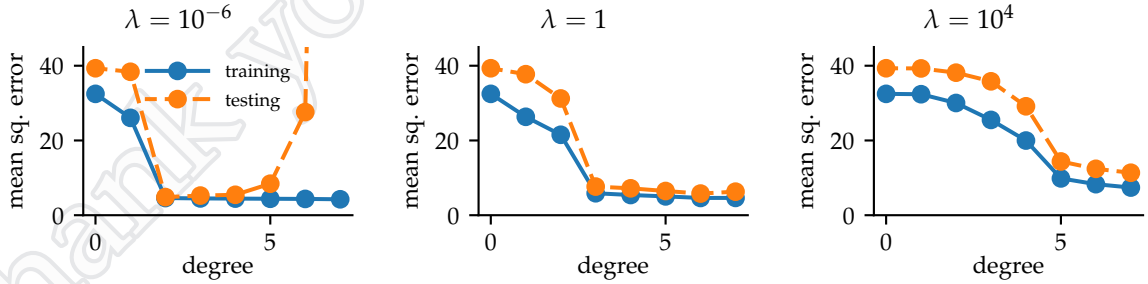

Figure 6.5: Effect of regularization on regression. Training and testing errors as a function of the degree of the constructed polynomial features. Training and testing data are the same as in Figure 6.4(middle). (left) small regularization strength; (middle) medium regularization strength; (right) large regularization strength.

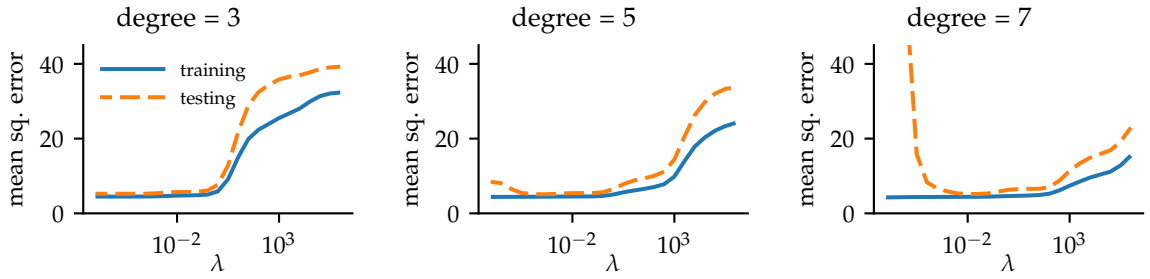

Figure 6.6: Effect of regularization on regression. Similar to Figure 6.5, but each plot is a fixed set of parameters, and errors are plotted as a function of regularization strength. (left) degree 3 polynomial terms as constructed features for a total of 10 features; (middle) degree 5 for a total of 21 features; (right) degree 7 for a total of 36 features.

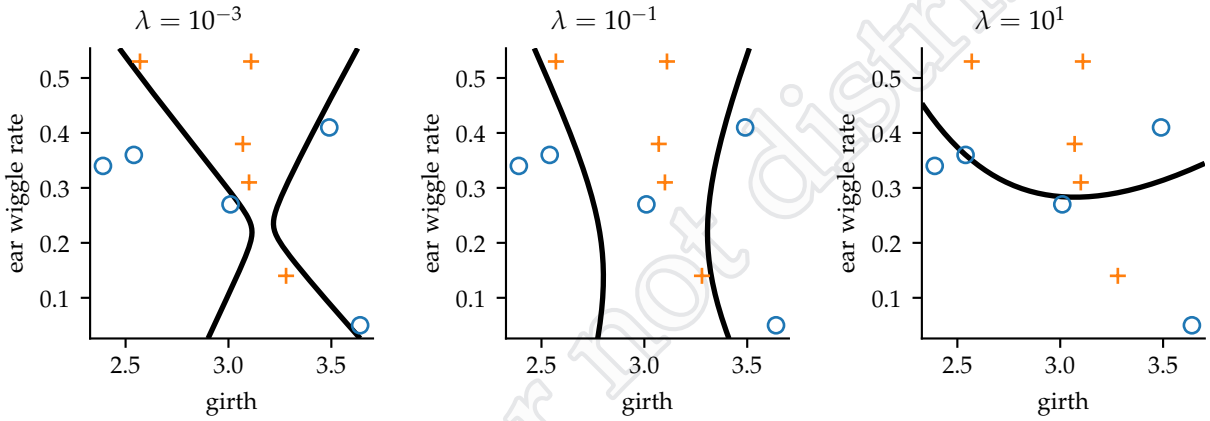

Figure 6.7: Effect of regularization on classification using fourth-degree polynomial as the discriminant. Decision surfaces after fitting the plotted data with logistic regression with different regularization strengths.

becomes more similar to the training error. Usually a middling regularization strength is best. But, determining the best regularization strength would require knowing the testing error, something that is not known at training time.

Figure 6.7 shows examples of using logistic regression to learn a fourth degree polynomial decision boundary with different regularization strengths. Again, we can see that as we increase the regularization strength, the resulting boundary becomes more smooth. As we decrease the regularization strength the training data are more accurately fit, but probably at the cost of overfitting and doing poorly on testing data.

Regularizers like ℓ_2 regularization treat all weights equally: Every weight is squared and all the results added together. This means all

weights should have roughly equal effects on the resulting learned function. If varying one weight has a large effect, but varying another weight has very little effect, regularization can produce strange results. For ease of explanation, we have ignored this critical aspect. We address normalization in Chapter 9. In real applications of regularization, some form of normalization should be applied.

Benefits of Regularization

We have traded the difficulty of selecting degree of the polynomial (or, more generally, how many extra constructed features to add) for the difficulty of selecting the regularization strength. Just as with the degree of the polynomial, we cannot tell from the training data what the right regularization strength is.

Selecting the regularization strength might seem no better than selecting the features to use. However, the regularization strength is a single real-valued quantity. By contrast, there are many possible features that could be added (the examples here just consider polynomials, but there are many other possible types of features). Optimizing over a single continuous value is simpler than optimizing over the combinatorially large space of possible features. In practice, this means that we often add many extra features that might be useful and let regularization "sort it out." The range of reasonable regularization strengths is often quite large. Consider Figure 6.6. The horizontal axis is logarithmic and in each plot there is a broad valley of good values. Thus, we do not need to find an exact right regularization strength, but rather only narrow down a good value to within a few *orders of magnitude*, a far easier task.

Further information
We often prefer to err on the side of adding too many extra features and use regularization to control the complexity of the resulting regressor or classifier.

Validation

Although the regularization strength might be easier to select than the number of types of extra features to construct, it still cannot be selected based on the training data: The training data are best fit when λ=0 because then the loss function only cares about the training fit. The regularization strength is an example of a **hyper-parameter**: a parameter of the learning method itself (not something produced by the learning method). The degree of the polynomial of the constructed features would be an example of another possible hyper-parameter. Even the choice of which machine learning method to use could be considered a hyper-parameter. Hyper-parameters specify how the learning algorithm will work. They are not part of the learned function (like parameters are), but control what function will be learned. They cannot be fit using training data because they

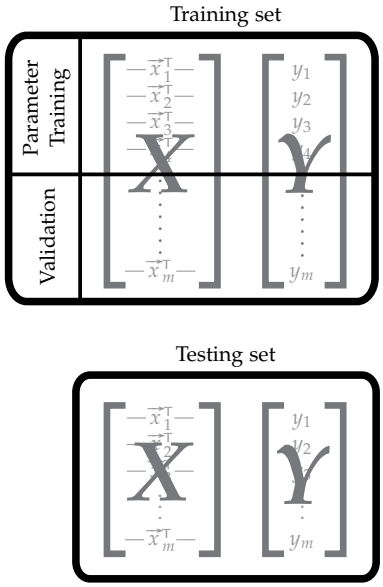

Figure 6.8: Structure for hold-out validation

control the learning algorithm itself.

But then, how to pick hyper-parameters? *If* we had testing data, we could pick the hyper-parameters based on the testing data. The testing data was not used in the generation of the weights and therefore can serve as an independent test of whether the *method* for generating the weights (and thus the hyper-parameters which specified the method) would produce weights that generalize well. Since we do not have testing data, we can approximate it by holding some of our training data in reserve to be used like testing data. This approach is called **hold-out validation**.

Hold-out validation works as shown in Figure 6.8. The training set is split into two parts: the **parameter training set** and the **validation set**. Training examples ($\vec{x}$-y pairs) are randomly shuffled and a certain fraction of them assigned to the parameter training set and the rest to the validation set. Typically more are assigned to the parameter training set (50%, 80%, and 90% are all common percentages used). Both new datasets (the parameter training set and the validation set) have their own $\boldsymbol{X}$ and $\boldsymbol{Y}$ matrices. Every row of $\boldsymbol{X}$ and the corresponding element of $\boldsymbol{Y}$ in the training set is placed in either the parameter training set or the validation set.

Once split, we use the validation set to check the performance of the learning algorithm run on the parameter training set with particular settings of the hyper-parameters. We try this for various hyper-parameter settings and select the best hyper-parameter. Algorithm 6.9 shows this general algorithm. Note that the *testing set plays no part in hold-out validation*. It is still used only to judge the accuracy of the final learned function, after all parameter and hyper-parameter tuning has been completed.

For the specific example of using hold-out validation to select the regularization strength of ridge regression, the procedure would work as follows. First, we select what fraction of examples to use for parameter training. We select 80%. Then, we select what set of regularization strengths to try. We select the set $\Gamma = \{10^{-2}, 10^{-1}, 1, 10, 10^2\}$. In Algorithm 6.9, Γ is the set of hyper-parameters to try and γ is any particular hyper-parameter. In this case, γ is the same as λ because we have only a single hyper-parameter called λ.

With these choices made, we are ready to run the hold-out validation algorithm. For this example, suppose we have 100 examples in our training set. We randomly divide them (keeping corresponding $\vec{x}$-y pairs together) into 80 examples in our parameter training set and 20 examples in out validation set. We have 5 hyper-parameter settings, so we learn 5 five times using ridge regression. Each time we use the same data (the 80 examples in the parameter training set), but we vary the regularization strength: The first time $\lambda = 10^{-2}$; the

HOLD-OUT VALIDATION
In: training, Γ
Out: γ^*

(ptrain,valid) $\leftarrow$ SPLIT(training)
besterr $\leftarrow \infty$
for all $\gamma \in \Gamma$ **do**
  $\vec{w} \leftarrow$ LEARN(ptrain,γ)
  verr $\leftarrow$ ERROR(valid,$\vec{w}$)
  if verr $<$ besterr **then**
    besterr $\leftarrow$ verr
    $\gamma^* \leftarrow \gamma$
return γ^*

Algorithm 6.9: Hold-out validation algorithm. SPLIT splits the data randomly into a parameter training set (ptrain) and a validation set (valid). LEARN is the core learning function. It takes a training set and the hyper-parameter setting and returns the learned weights. ERROR computes the error (or total per-item loss) on a dataset (first argument) for the function specified by the weights (second argument). Γ is a set of hyper-parameter values to try.

next time $\lambda = 10^{-1}$; and so on. Each learning iteration produces a different weight vector, $\vec{w}$. Each weight vector is used to predict the targets for each of the examples in the validation set. We compare these predictions to the correct values in the validation set and compute the total error (sum of the squared errors). Note that this does *not* include the regularization term of the loss; it is just the raw error on the validation data. We call this the validation error (denoted "verr" in the algorithm). We return the value of λ that resulted in the best (smallest) validation error. This is our choice for the hyper-parameter.

Notes on Validation

Validation is more general than just picking regularization strength. It can be used to select any hyper-parameter. Or, even combinations of hyper-parameters. For instance, we could search for the best setting of both the regularization strength and the degree of the polynomial. In such cases, we have to try every combination of both parameters. So, if we wanted to try regularization strengths of $\{10^{-2}, 10^{-1}, 1, 10, 10^2\}$ and polynomial degrees of $\{1, 2, 3\}$, we would have 5×3=15 combinations to try. While 15 is reasonable, for more realistic situations, we might be trying 10 to 100 possible values for each hyper-parameter. For more than two or three hyper-parameters, it is not feasible to do such an exhaustive search. Other methods (beyond the scope of this text) are possible; they keep the same parameter-training and validation split, but do not use an exhaustive loop.

For classification, there are a few commonly employed variations. We, generally, do not use the same error (or loss) function for checking against the validation set that we do when learning. For example, in logistic regression, the per-item loss is $l(y, f) = -\ln \sigma(yf)$. This is a smooth loss function and works well for minimization. However, when checking the error on the validation set, we do not need a smooth loss function. Therefore, in classification, the ERROR function in Algorithm 6.9 typically just counts the number of examples misclassified.

Additionally, if the classes are imbalanced, we usually split the data to make sure that the same class imbalance is maintained in the parameter training set and in the validation set. This is done by splitting the data associated with each class separately. This is called **stratified sampling** (or splitting).

Once the hyper-parameters have been found, we still do not have a parameter setting (note that Algorithm 6.9 does not return parameters). To select parameters, the hyper-parameters found with

Further information

A dataset has **class imbalance** if at least one of the classes is significantly under-represented, compared with an even distribution of classes. For instance, if a training set of 1,000 hippopotamuses had 30 with y = aggressive and 970 with y = passive, this data would be considered to have class imbalance, even if this skewed distribution was representative of the true label fraction.

validation are fixed, but now the learning algorithm is run on the entire training set. This produces the final output of the entire learning system. It is only at this point, after hyper-parameters and parameters have been selected (and those selections not revisited), that the testing data is employed to judge the quality of the entire method.

Cross-Validation

In hold-out validation, each example from the training set serves either as a parameter-training example or as a validation example. However, especially in small training sets or when there is extreme class imbalance, this can cause problems if a particularly informative (or misleading) example is included in the training set but a corresponding example is not included in the validation set. **Cross-validation** (CV), also called k-fold cross-validation, lets every example serve both as a parameter-training example and as a validation example. For this reason, it tends to be more popular in machine learning, although it does require more computational effort.

Figure 6.10 shows the structure of cross-validation. We split the training data into k equal-size "folds." For each hyper-parameter setting, we try each fold as a validation set (and the other $k-1$ folds as the parameter training set). We add the errors across all folds as the validation error of the hyper-parameter setting. Algorithm 6.11 shows this as an algorithm.

Leave-one-out cross-validation (LOOCV) is an extreme form of cross-validation where k=m. That is, training example is its own fold. For some learning algorithms, it is possible to compute the LOOCV error without training on each of the m folds separately. This can make it an attractive version of cross-validation.

Doing cross-validation requires k times as much computational effort (roughly) as hold-out validation. In any type of validation, the learning (and error checking) can be done in parallel across different hyper-parameter values. In cross-validation, these computations can also be done in parallel across the different choices of folds to use as the validation set. Depending on the computational resources available and the parallelism possible in the learning algorithm itself, this can speed up hyper-parameter selection by validation. Generally, cross-validation is preferred to hold-out validation because it uses all of the data in both training and validation.

Cross-validation has a "hyper-hyper-parameter:" k. We do not continue to nest hyper-parameter selection methods to select k. Rather, k is chosen to match computational and data size limits. Both large and small values of k have (different) problems in estimating the true best value of the hyper-parameter(s). Small values of k use a

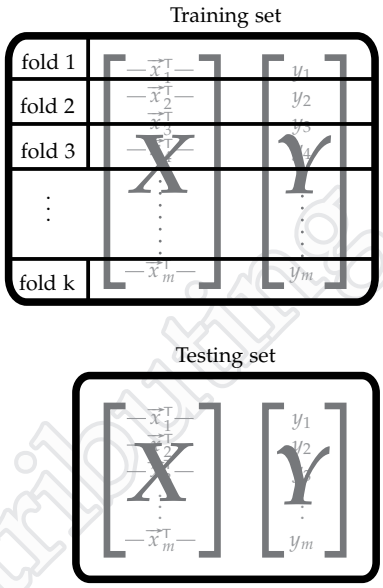

Figure 6.10: Structure for cross-validation

CROSS-VALIDATION
In: training, k, Γ
Out: γ^*

$(f_1, \ldots, f_k) \leftarrow$ SPLIT-K(training)
besterr $\leftarrow \infty$
for all $\gamma \in \Gamma$ **do**
  verr $\leftarrow 0$
  for all $j \in \{1, 2, \ldots, k\}$ **do**
    $\vec{w} \leftarrow$ LEARN(training$-f_j, \gamma$)
    verr $\leftarrow$ verr + ERROR($f_j, \vec{w}$)
  if verr $<$ besterr **then**
    besterr $\leftarrow$ verr
    $\gamma^* \leftarrow \gamma$
return γ^*

Algorithm 6.11: Cross-validation algorithm. SPLIT-K splits the data randomly and evenly into k folds. f_j is the jth fold; training$-f_j$ is all of the training data except the jth fold.

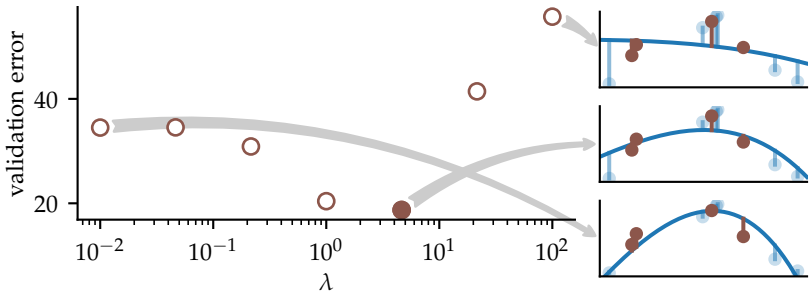

Figure 6.12: Example of hold-out validation to select the regularization strength for ridge regression

(relatively) large fraction of the data as validation set(s) and therefore train on datasets that are significantly smaller than the total training set that will eventually be used to select $\vec{w}$. Large values of k produce overly confident estimates of the validation error, because they are based on only a few examples. The exact correct balance depends on the problem and the amount of data in the training set. Typically k is chosen to be 5 or 10. As a "hyper-hyper-parameter," its selection is not as critical to the overall performance as other things (such as the choice of features or the core learning method).

Further information
Why 5 or 10? Probably because that is the number of fingers most people have on 1 or 2 hands.

Hold-out Validation Example

We will apply linear regression with ℓ_2 regularization (ridge regression) to the running example (Table 2.1), but only using the first feature (girth) to predict y (speed). While cross-validation might be more common in practice, hold-out validation is more straightforward, so we will use it in this example to illustrate. We choose a fourth-degree polynomial fit (and thus, there are five constructed features). We select seven, logarithmically evenly spaced values for λ. We choose 40% of the data in the validation set (a total of 4 out of the 10 points), leaving 60% for parameter learning (6 of the 10 points).

Figure 6.12 shows the computations in using hold-out validation, graphically. The left side shows the computed validation errors for the seven selected hyper-parameter possibilities. The filled-in circle represents the final chosen value of λ: 4.6. On the right side are three examples of how these values were calculated. The (light) blue points are the parameter training data. They were used to learn the blue function. The validation error was calculated by comparing the predictions to the true values on the validation set, shown in brown. The lengths of the brown vertical lines are the errors. The total validation error is the sum of the squares of these lengths.

If we were to have used cross-validation, the validation error

would have been calculated from multiple blue-brown splits of the data. That is, there would have been k small plots on the right side for each point on the left plot. The validation error would be the sum of the errors from each of these k folds.

After selecting λ, the next step would be to use this regularization strength with then entire training dataset to find the final weight vector. This weight vector is then used for prediction. Note that because λ is a hyper-parameter and not a parameter, it is not required for prediction on future or testing data.

Exercises

Exercise 6.1

The original dataset has n raw features. Consider a constructed dataset has all possible polynomial features up to (and including) those of degree d. Note that x_1x_2 is the same as x_2x_1. Let p be the total number of constucted features.

a. What is p as a function of n and d?
b. Write code to generate the features. Assume input of an m-by-n matrix and the value for d. Your code should output an m-by-p matrix.

Exercise 6.2

Redo the linear regression fit from Exercise 4.2, but with ridge regression. (Again, use a linear algebra library or software to perform the calculations.)

a. What is the learned weight vector when $\lambda = 0$?
b. What is the learned weight vector when $\lambda = 10$ if you penalize the intercept weight?
c. What is the learned weight vector when $\lambda = 10$ if you do not penalize the intercept weight?

Exercise 6.3

What if we would like to apply a different regularization strength to each parameter (ignoring how feasible it might be to set all of these hyper-parameters)? That is, the regularizer would be

$$R(\vec{w}) = \sum_{j=1}^{n} \lambda_j w_j^2 \, .$$

If this regularizer (which does not need to be multiplied by λ because the strengths are already built into R) were used in linear least squares, what would be the formula to find the $\vec{w}$ that minimizes the loss (including the regularizer)?

Exercise 6.4

Which of the following functional forms (hypothesis spaces) can be fit using linear least-squares regression, possibly with constructed features? In each case, the parameters are $\{w_1, w_2, \dots\}$ and $\vec{x}$ is two-dimensional. If the functional form can be fit, provide the feature mapping, ϕ.

a. $f(\vec{x}) = e^{w_1 x_1} + e^{w_2 x_2}$
b. $f(\vec{x}) = w_1 e^{x_1} + w_2 e^{x_2}$
c. $f(\vec{x}) = w_1 e^{w_2 x_1} + w_3 e^{w_4 x_2}$
d. $f(\vec{x}) = w_1 e^{x_1} + w_2 e^{x_2} + w_3 x_1 + w_4 x_2$

e. $f(\vec{x}) = w_1 x_1 + w_2 x_2$
f. $f(\vec{x}) = w_1 x_1^2 + w_2 x_2^2$
g. $f(\vec{x}) = w_1 x_1^2 + w_2 x_2^2 + w_3 x_1 x_2 + w_4$
h. $f(\vec{x}) = w_1 (x_1^2 + x_2^2) + w_2$
i. $f(\vec{x}) = w_1 x_1 + w_2 w_3 x_2$
j. $f(\vec{x}) = w_1 x_1 + x_2$

Exercise 6.5

Consider using holdout validation to select the regularization strength for ridge regression. Figure 6.13 shows the error on the parameter training set, validation set, and testing set, as a function of the regularization strength, λ. The circle indicates the value of the regularization strength chosen by holdout validation.

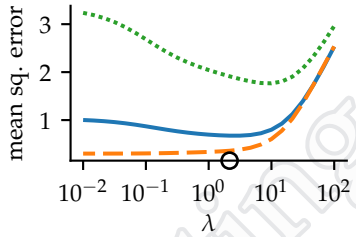

Figure 6.13: Plot to be labeled for Exercise 6.5

a. Which curve is the parameter training error? Why?
b. Which curve is the validation error? Why?
c. Which curve is the testing error? Why?
d. Based on this plot, which would be the best choice for λ? Why?
e. Why did we not pick this value for λ?

Exercise 6.6

Consider using ℓ_p regularization (instead of ℓ_2):

$$R(\vec{w}) = \sum_{j=1}^{n} |w_j|^p \ .$$

When $p = 2$, this is ℓ_2 regularization. However, we can vary p (a hyper-parameter) to change the effects.

a. What is the gradient descent update rule for linear least squares using ℓ_p regularization?
b. What is the gradient descent update rule for logistic regressionusing ℓ_p regularization?
c. Why might gradient descent be problematic for $p \leq 1$?

Exercise 6.7

For ridge regression, it is possible to calculate the total leave-one-out cross-validation error without relearning the weight vector for each of the m folds. Specifically, we need only calculate the inverse $(\boldsymbol{X}^\intercal \boldsymbol{X} + \lambda \boldsymbol{I})^{-1}$ once (for the full dataset).

To demonstrate this, prove that the leave-one-out cross-validation error for ridge regression can be calculated as

$$\sum_{i=1}^{m} \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2$$

where

$$\begin{aligned}
\boldsymbol{B} &= (\boldsymbol{X}^\mathsf{T}\boldsymbol{X} + \lambda \boldsymbol{I})^{-1} \\
\vec{w} &= \boldsymbol{B}\boldsymbol{X}^\mathsf{T}\boldsymbol{Y} && \text{the weights learned on the full dataset} \\
\hat{y}_i &= \vec{x}_i{}^\mathsf{T}\vec{w} && \text{the prediction on the training point } i \\
h_i &= \vec{x}_i{}^\mathsf{T}\boldsymbol{B}\vec{x}_i && \text{which is the } i\text{th diagonal element of } \boldsymbol{X}\boldsymbol{B}\boldsymbol{X}^\mathsf{T}
\end{aligned}$$

Hints:

1. The Woodbury matrix identity is helpful. The specific form that helps with this problem is

$$(\boldsymbol{A} - \vec{v}\,\vec{v}^\mathsf{T})^{-1} = \boldsymbol{A}^{-1} + \boldsymbol{A}^{-1}\frac{\vec{v}\,\vec{v}^\mathsf{T}}{1 - \vec{v}\,\boldsymbol{A}^{-1}\vec{v}}\boldsymbol{A}^{-1} \,.$$

2. If $\boldsymbol{X}_{-i}$ is the dataset without data point (row) i, then

$$\boldsymbol{X}_{-i}^\mathsf{T}\boldsymbol{X}_{-i} = \boldsymbol{X}^\mathsf{T}\boldsymbol{X} - \vec{x}_i\vec{x}_i{}^\mathsf{T}$$

Exercise 6.8

This exercise concerns binary classification when the original (raw) data has two features. It explores using a linear classifier on constructed features.

a. Demonstrate that any classifier (in the original raw features) that corresponds to a circle centered at the origin can be represented as a linear classifier in the constructed feature mapping $\phi(\vec{x}) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_1x_2 \\ x_2^2 \end{bmatrix}$.

b. What feature mapping should be used if we want to *only* allow classifiers that correspond to circles centered at the origin?

c. What about a mapping that allows only circles, but centered anywhere in the plane?

d. Prove there is no feature mapping that only allows circles where the positive class is inside and the negative class is outside (and does not permit hypotheses where the inside is negative and the outside is positive).

e. Why is there no mapping that allows ellipses but not hyperbolas?